

CS582: MACHINE LEARNING

Lab 6 — Unsupervised Learning Solutions
Department of Computer Science

1. Purpose and Effect of the Neighborhood Function in SOM

The Self-Organizing Map (SOM) is an unsupervised neural network that maps high-dimensional input spaces onto low-dimensional (usually 2D) topological structures. Unlike standard competitive learning networks where only the winning neuron learns, SOM uses a neighborhood function to allow surrounding neurons to learn as well.

Primary Purpose

The primary purpose of the neighborhood function $h(nb, t)$ is to define a spatial region around the Best-Matching Unit (BMU or winning neuron nb) within which nearby neurons are also updated. This cooperative learning strategy forces neurons that are physically close on the grid to develop sensitivity to similar input patterns.

Learning Dynamics and Effects

- **Topological Preservation:** By updating a cluster of adjacent neurons together, the grid organizes itself such that the relative spatial ordering of the high-dimensional input space is preserved. Points that are close in the input space will map to close coordinates on the SOM grid.
- **Global-to-Local Convergence:** The neighborhood size is typically initialized to be large, covering almost the entire network. This allows for rapid global organization early in the training process. Over time (as t increases), the neighborhood size decreases (usually using an exponential decay rate). This shifts the network from global ordering to local fine-tuning, allowing individual neurons to specialize in representing detailed sub-features without disrupting the global map layout.

2. SOM-Based Intruder Detection System Design

An intrusion detection system (IDS) can be modeled as an anomaly detection system using a SOM trained on legitimate user behavior data. The network uses four main characteristics: (i) login time, (ii) login duration, (iii) program types, and (iv) number of programs run.

Training Process

To train the SOM classifier, we record multi-dimensional feature vectors representing normal user sessions. Each vector contains normalized values of the four characteristics. During training, these vectors are sequentially fed into the SOM. For each session vector, the network

identifies the BMU using the minimum Euclidean distance. The weights of the BMU and its neighbors are updated, moving them closer to the input vector. Over many iterations, the SOM organizes into a grid of 'normal' user behavior profiles.

Data Preprocessing and Normalization

Normalization is mathematically essential. The raw input features have vastly different ranges (e.g., login times are 0–24 hours, whereas login durations can be minutes or hours, and program counts could range from zero to hundreds). Left unnormalized, features with larger numerical values will dominate the distance calculations, making the other features negligible. We normalize both inputs and weights to put them on a uniform scale (e.g., mapping onto a unit hypersphere), ensuring each characteristic contributes equally to the clustering process.

Data Volume and Network Size

- **Data Volume:** We require a substantial history of legitimate user logs (e.g., several weeks or months of typical activity) to cover weekly cycles, shift changes, and standard behavioral variation across different user roles.
- **SOM Grid Size:** The size of the SOM must be balanced. If the network is too large, it will overfit and represent exact, individual sessions instead of generalizing. If it is too small, it will overgeneralize and fail to differentiate distinct user categories. The optimal grid size should be determined experimentally (e.g., starting with a 10x10 grid) and selecting the size that yields the lowest quantization error on validation data.

System Effectiveness

This system is highly effective for detecting blatant anomalies. Because normal users consistently fall into predictable clusters, any session showing unusual behavior (e.g., an unauthorized login at 3:00 AM running dozens of strange programs) will produce a feature vector that is highly distant from all normal BMUs on the grid. This high quantization error triggers an immediate intrusion alert.

However, the system has a critical limitation: it is vulnerable to sophisticated, slow attacks. If an intruder deliberately mimics the behavior profile of an established normal user cluster, the SOM will map the intruder's session to that normal node, failing to raise an alert.

3. Bank Fraud Detection Using Competitive Learning

A bank can utilize competitive learning (using a winner-take-all neural network) to cluster credit card transaction data and flag potential fraud.

Methodology and Fraud Detection

Each transaction is represented by a vector containing variables like transaction amount, shop category, time, and date. During training, the competitive learning network clusters these vectors into typical spending profiles. If a credit card is stolen, subsequent transactions will likely differ from the customer's established clusters. Any new transaction that falls beyond a

threshold Euclidean distance from the customer's typical centroids is flagged as an anomaly, allowing the card to be suspended automatically.

Handling Data Imbalance

There is a massive data imbalance because normal transactions outnumber stolen transactions by orders of magnitude. If trained directly on raw data, the neural network's weights will be completely pulled towards the massive volume of normal transactions, and fraud patterns will either be lost as noise or merged into normal behavior groups.

To mitigate this, we can implement the following strategies:

- **Unsupervised Anomaly Detection:** Instead of trying to classify 'normal vs. fraud' directly, we configure the competitive network to only learn and define the boundaries of normal behaviors. Any transaction falling outside these high-density normal boundaries is classified as fraudulent.
- **Random Sampling:** Before training, we can apply random sampling on the legitimate transactions. Training on small, diverse, randomly sampled subsets helps prevent the network from being overwhelmed and ensures that less common but normal behaviors are still represented.

4. K-Means Clustering Detailed Numerical Problem

We are given a dataset consisting of scores on two variables (A and B) for seven individuals. The goal is to group them into two clusters using the K-Means clustering algorithm.

Subject	Score A	Score B
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Initialization

The initial cluster centroids (means) are defined by the two individuals furthest apart using the Euclidean distance measure:

- Group 1 Centroid (μ_1): Subject 1 = (1.0, 1.0)

- Group 2 Centroid (u2): Subject 4 = (5.0, 7.0)

Iteration 1

1. Assignment Step: Calculate Euclidean distance from each subject to both centroids u1(1.0, 1.0) and u2(5.0, 7.0).

Subject	Distance to u1	Distance to u2	Assignment
S1 (1.0, 1.0)	$d = 0.0$	$d = \sqrt{((1-5)^2 + (1-7)^2)} \approx 7.21$	Group 1
S2 (1.5, 2.0)	$d = \sqrt{((1.5-1)^2 + (2-1)^2)} \approx 1.12$	$d = \sqrt{((1.5-5)^2 + (2-7)^2)} \approx 6.10$	Group 1
S3 (3.0, 4.0)	$d = \sqrt{((3-1)^2 + (4-1)^2)} \approx 3.61$	$d = \sqrt{((3-5)^2 + (4-7)^2)} \approx 3.61$	Group 1 (tie-breaker)
S4 (5.0, 7.0)	$d = \sqrt{((5-1)^2 + (7-1)^2)} \approx 7.21$	$d = 0.0$	Group 2
S5 (3.5, 5.0)	$d = \sqrt{((3.5-1)^2 + (5-1)^2)} \approx 4.72$	$d = \sqrt{((3.5-5)^2 + (5-7)^2)} = 2.50$	Group 2
S6 (4.5, 5.0)	$d = \sqrt{((4.5-1)^2 + (5-1)^2)} \approx 5.32$	$d = \sqrt{((4.5-5)^2 + (5-7)^2)} \approx 2.06$	Group 2
S7 (3.5, 4.5)	$d = \sqrt{((3.5-1)^2 + (4.5-1)^2)} \approx 4.30$	$d = \sqrt{((3.5-5)^2 + (4.5-7)^2)} \approx 2.92$	Group 2

2. Centroid Update Step: Re-calculate the mean vector of each group.

- New u1 (from S1, S2, S3): $((1.0 + 1.5 + 3.0)/3, (1.0 + 2.0 + 4.0)/3) = (1.83, 2.33)$
- New u2 (from S4, S5, S6, S7): $((5.0 + 3.5 + 4.5 + 3.5)/4, (7.0 + 5.0 + 5.0 + 4.5)/4) = (4.13, 5.38)$

Iteration 2

1. Assignment Step: Re-evaluate distances with the updated centroids u1 (1.83, 2.33) and u2 (4.13, 5.38).

- **S1, S2:** Remain closest to u1. (Group 1)
- **S3 (3.0, 4.0):** $d(S3, u1) \approx 2.04$ and $d(S3, u2) \approx 1.78$. Since $1.78 < 2.04$, S3 moves to Group 2.
- **S4, S5, S6, S7:** Remain closest to u2. (Group 2)

2. Centroid Update Step: Re-calculate the mean vector of each group.

- New u_1 (from S1, S2): $((1.0 + 1.5)/2, (1.0 + 2.0)/2) = (1.25, 1.50)$
- New u_2 (from S3, S4, S5, S6, S7): $((3.0 + 5.0 + 3.5 + 4.5 + 3.5)/5, (4.0 + 7.0 + 5.0 + 5.0 + 4.5)/5) = (3.90, 5.10)$

Iteration 3

1. Assignment Step: Re-evaluate distances with the updated centroids u_1 (1.25, 1.50) and u_2 (3.90, 5.10).

- **S1, S2:** Remain closest to u_1 . (Group 1)
- **S3 (3.0, 4.0):** $d(S3, u_1) \approx 3.05$ and $d(S3, u_2) \approx 1.42$. Remains closest to u_2 . (Group 2)
- **S4, S5, S6, S7:** Remain closest to u_2 . (Group 2)

2. Convergence: Since no subjects shifted their group assignments during Iteration 3, the algorithm has successfully converged. Training stops here.

Final Classification Output

Cluster Group	Assigned Subjects	Final Centroid
Group 1	S1, S2	(1.25, 1.50)
Group 2	S3, S4, S5, S6, S7	(3.90, 5.10)